

2025-1학기 안드로이드 프로그래밍 기말 프로젝트

# 공공 클래스 앱 개발 보고서 온클래스 [ON-Class]

과목명 : 안드로이드 프로그래밍

담당교수 : 구민정 교수님

팀원 : 장의엽(20213211)  
최지환(20203525)

팀명 : 2비트



ANDROID PROGRAMING

# CONTENTS



**01 / 프로젝트 개요** | 주제선정 배경  
디자인 리서치

**02 / 개발 앱 소개** | 개발 앱 주제  
유사 앱 비교

**03 / 와이어프레임** | 디자인 스케치 및  
기능 소개

**04 / 프로토타이핑** | 최종 앱 소개  
주요 코드 소개

**05 / L&L** | AI 도움 사항  
느낀점 및 후기

## 프로젝트 개요 Project Overview

### 주제선정 배경

### ‘갓생시대’ 자기계발이 경쟁력이다!

2023년 HRE테크 기업 인크루트에서 직장인 805명을 대상으로 조사한 설문자료에 의하면 **본업 외에 자기계발을 하는 직장인이 75.2%를 기록함** 이는 잡코리아에서 2019년 직장인 1,394명을 대상으로 실시한 자기계발 설문조사 수치인 **50.3%보다 높은 수치로 점차 증가하는 것을 알 수 있음**

2021년 인터넷 강의 기업 에듀윌에서 직장인, 취준생을 891명을 대상으로 조사한 자기계발 활동 관련 설문조사에서 **직장인 및 취업준비생 90.1%가 자기계발의 필요성을 느낀다고 응답함**

2022년 시장조사 전문기업 엠브레인 트렌드모니터에서 성인남녀 1,000명을 대상으로 조사한 결과 **‘스스로의 가치 향상’에 대해 관심이 있다**에 20대는 76.8%, 30대는 63.2%를 보였으며 이유로는 **‘자기계발을 하지 않으면 도태되기 쉬운 사회’로 80.1%의 결과를 보여줌**

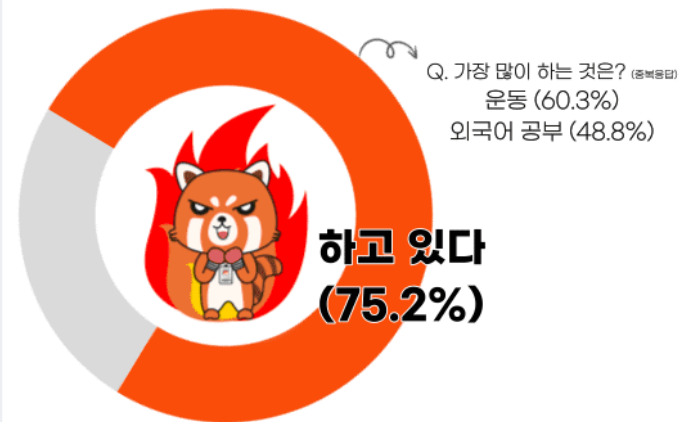


**현재 대한민국 20대, 30대 대다수가 자기계발이 필요하다 느끼며 이를 인지하는 사람이 증가하는 추세임**

**“나는 갓생러”,  
직장인 10명 중 8명 본업 외 자기 계발 한다**

조사대상 : 직장인 805명 | 조사기간 : 23년 10월 25일~30일(엿새간)

Q. 본업 외에 자기 계발을 하고 있나요?



Incruit

데이터 출처 : 인크루트 "직장인 자기계발 설문조사"

## 프로젝트 개요 Project Overview

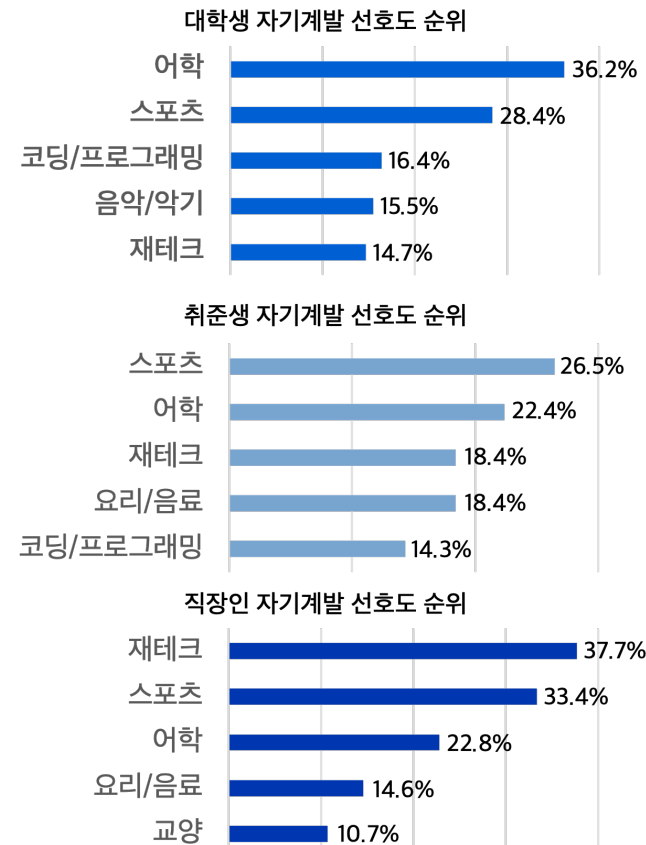
### 자기계발, 무엇을 위해 하는가?

트렌드 리포트 오픈서베이에서 성인 2,000명을 대상으로 조사한 설문 데이터에 의하면 **대학생, 취준생, 직장인**을 기준으로 **TOP 5** 안에 공통적으로 들어가는 자기계발은 **스포츠, 어학, 재테크**이며 그 외에도 **코딩/프로그래밍, 요리/음료**가 높은 선호도를 보임

그 외에도 **온라인 자기계발 앱(클래스101, 탈잉 등)**에서 **TOP 10**을 차지하는 클래스의 장르는 **어학, 드로잉, 영상 편집, 포토샵, 쿠킹클래스, 자격증** 등 **생산적이면서 수익을 창출**해낼 수 있는 클래스를 배우길 선호한다는 것을 알 수 있음

이는 사람들의 자기계발의 목적이 **취업이나 이직에 유리**할 수 있도록 하거나 기존의 자신의 업 외에 **투잡에 활용**하기 좋은 **생산적 기술**을 배우는 등 **자신의 가치를 높이는 것을** 목표로 한다는 사실을 알 수 있음

**➡ 자기계발 = 수익 창출**  
**사람들의 자기계발 선호도는 결국 수익을 내기 위함**



데이터 출처 : 트렌드 리포트 오픈서베이 "자기계발 트렌드"

# 프로젝트 개요 Project Overview

## 자기계발 현 실태, 온라인 클래스

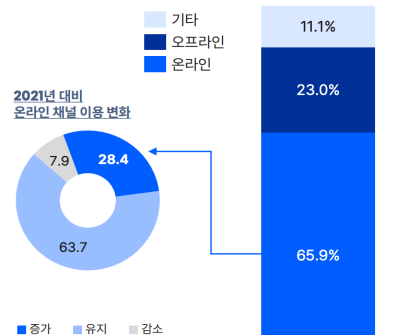
트렌드 리포트 오픈서베이의 데이터에 의하면 자기계발을 위해 온라인을 활용하는 유저가 65.9%로 높은 비율을 차지하는 것을 알 수 있으며 이는 내가 원하는 시간 및 장소에 맞추어 볼 수 있으며 어려운 부분을 다시 볼 수 있다는 효율성과 간편함으로 인하여 생긴 선호도임

대다수의 사람들은 유료 플랫폼이나 사설 강의 등을 의존 및 이용하고 있으나 유료 플랫폼으로만 이용할 의향이 있는 사람은 0.1%로 무료 콘텐츠라 할지라도 콘텐츠의 종류, 퀄리티에 따라 무료 클래스를 이용하고자 하는 의향이 64.6%로 매우 높은 양상을 보임

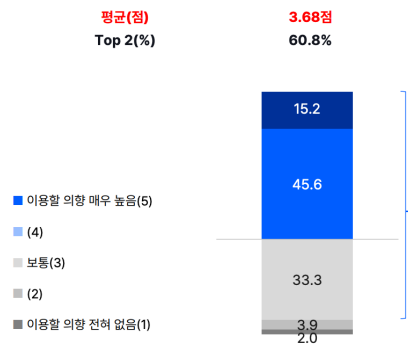


**바쁜 현대인들에겐 시공간의 제약이 없는 온라인 클래스가 인기 있으며 자기계발의 주요 수요가 2030이기에 경제적 부담을 적게 자기계발을 하는 것이 중요**

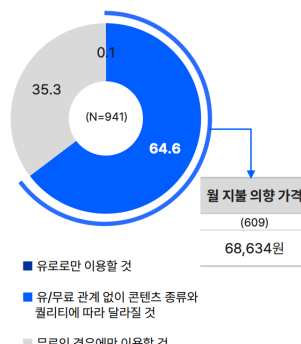
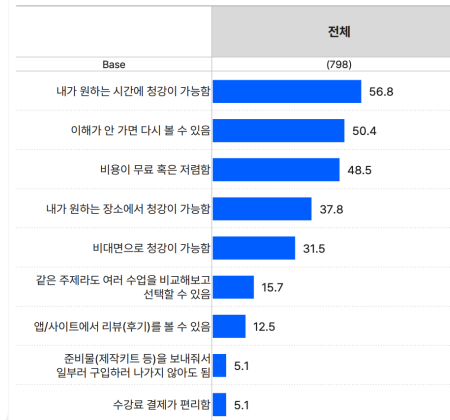
취미/자기계발 활동 시 주 이용 채널  
[Base: 전체 응답자, N=1000, 순위형 응답(1순위), %]



향후 1년 내 취미/자기계발 활동 시 온라인 채널 이용 의향  
[Base: 전체 응답자, N=1000, 척도형 응답(5점)]



온라인 채널 이용 시 만족점



데이터 출처 : 트렌드 리포트 오픈서베이 "자기계발 트렌드"

## 프로젝트 개요 Project Overview

### 자기계발, 무료로 할 수는 없을까?

2021년 국가평생교육통계조사에 의하면 2021년 기준 교육청 관할 평생교육기관 등에서 진행된 평생교육 프로그램 수는 약 22만 5천개이고 이는 전년 약 28만개 대비 20% 감소된 수치, 1개월 이상 운영된 프로그램 수는 16만개로 전년 19만개 대비 18% 감소된 수치임

공공 클래스의 감소 요인중 하나는 수요라 볼 수 있는데 이는 참여율 그래프를 통해 알 수 있음, 총 학습자 수는 전년인 2020년 대비 15.2% 감소하였고 특히나 오프라인 클래스 학습자의 경우 전년인 2020년 대비 60%나 감소됨

이를 통해 알 수 있는 건 공공기관에서 진행되는 무료 교육 프로그램은 다양한 기관으로 인한 개별 홍보의 부족, 낮은 접근성, 복잡한 신청 절차 등으로 인해 참여의 어려움이 존재하여 수요가 점차 줄어들고 있으며 또한 오프라인 클래스의 감소는 시공간의 제약이 존재한다는 점에서 점차 사라질 것으로 보임



**공공기관에서 운영되는 프로그램은 많으나  
여러가지 이유들로 인하여 참여가 어려움**



데이터 출처 : 2021년 국가평생교육통계조사(교육부)

## 프로젝트 개요 Project Overview

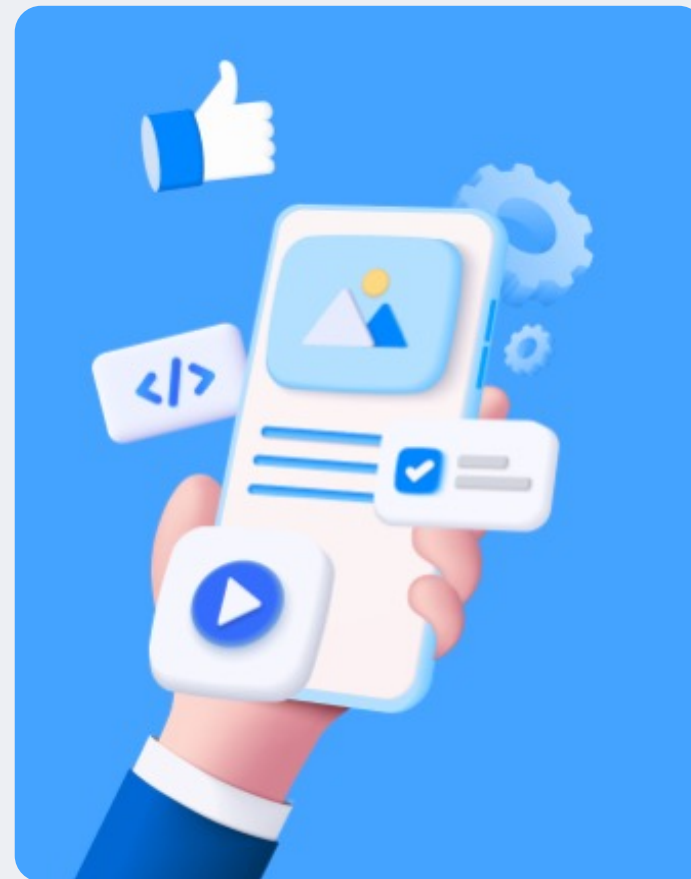
### 공공 클래스, 하나로 통합해보자!

이러한 문제점은 국가에서 운영하는 공공 클래스를 하나로 통합하여 시청할 수 있는 플랫폼이 존재한다면 이러한 어려움들을 해결할 수 있을 것으로 보임

기존 공공기관 클래스의 단점들인 복잡성(기관이 모두 다름)과 낮은 접근성(어떻게 무료 클래스를 들을 수 있는지 모름), 신청절차의 어려움(신청 과정, 홈페이지가 다양)의 문제들을 해결하기 위하여 공공 클래스를 앱 하나로 통합해 앱 내에서 간편하게 신청하고 시청까지 가능하게 하는 것을 목표로 함

이를 통해 대중은 쉽게 무료로 자기계발 클래스를 들을 수 있으며 플랫폼 이용자 수의 증가에 따라 자연스레 더 많은 클래스 제작과 일자리 창출 또한 기대할 수 있을 것으로 보임

**➡ 공공기관에서 운영하는 클래스를 하나의 앱으로 통합하여 접근성을 높이고 수요의 증가에 맞춰 더 다양한 클래스 제작을 노릴 수 있음**



이미지 출처 : hectodata Blog



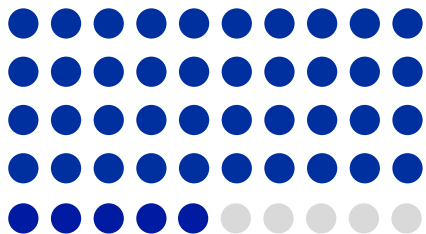
# 프로젝트 개요 Project Overview

## 디자인 리서치(설문조사)

- 기간 : 2025.04.31~05.12일 / 대상 : 20대 남녀 50인 / 품 : 네이버폼 설문조사

Q1.

자기계발을 해본 적이 있으십니까? (답변 총 50개)

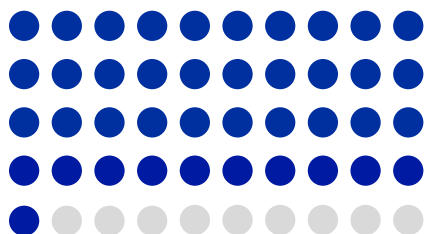


자기계발 해본 적이 있다(O)  
90% (45인)

자기계발 해본 적이 없다(X)  
10% (5인)

Q2.

자기계발을 위해 어떤 플랫폼을 주로 이용하시나요?  
(답변 총 50개)



온라인 플랫폼  
81% (41인)

오프라인 플랫폼  
21% (9인)



설문자 **대다수(90%)**가  
자기계발을 해보았으며  
그 중 **과반수 이상(81%)**이  
온라인 플랫폼을 이용해봄



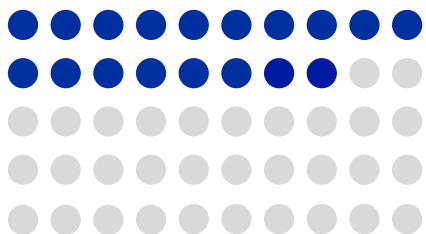
# 프로젝트 개요 Project Overview

## 디자인 리서치(설문조사)

- 기간 : 2025.04.31~05.14일 / 대상 : 20대 남녀 50인 / 품 : 네이버폼 설문조사

Q3.

공공기관에서 진행하는 자기계발 클래스에 대한 정보를 접해본 적이 있나요? (답변 총 44개)

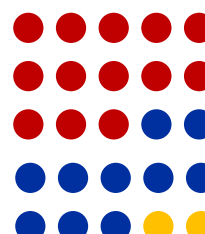


접해본 적이 있다(O)  
36% (18인)

접해본 적이 없다(X)  
64% (32인)

Q4.

접해본 적이 있는 경우 어떻게 접하게 되셨나요?  
(중복 응답 / 답변 총 25개)



직접 찾아본 경우 52% (13개)

온라인 홍보(앱 광고, 유튜브 등) 40% (10개)

현수막 홍보(현수막, 전단지 등) 8% (2개)

지자체 홈페이지 (공지, 광고 등) 0% (0개)



**설문자 과반수 이상(64%)이 공공 클래스 정보를 접해본 적이 없으며  
혹여나 정보를 접해본 경우 직접 찾은 경우가 52%를 기록함**

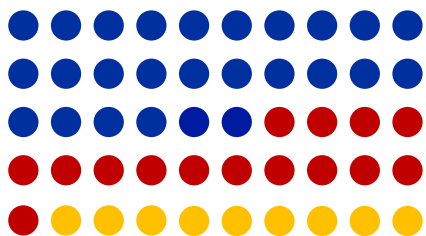
# 프로젝트 개요 Project Overview

## 디자인 리서치(설문조사)

- 기간 : 2025.04.31~05.14일 / 대상 : 20대 남녀 50인 / 품 : 네이버폼 설문조사

Q5.

공공 클래스의 접근이 어려운  
사유가 무엇이라 생각하시나요?  
(답변 총 50개)



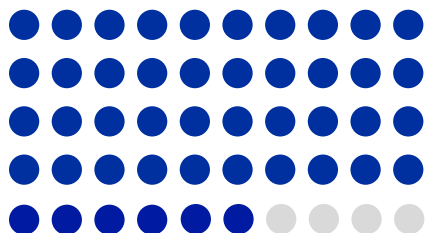
홍보의 부족성  
52% (26인)

시공간의 제약성  
30% (15인)

이용의 복잡성  
18% (9인)

Q6.

온클래스 앱이 나온다면  
사용하실 의향이 있으신가요?  
(답변 총 50개)



사용할 의향이 있다(O)  
92% (46인)

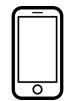
사용할 의향이 없다(X)  
8% (4인)



공공 클래스 접근이 어려운  
이유는 **홍보의 부족성**이  
가장 크며(52%)  
플랫폼이 출시될 경우 설문  
자 **대다수(92%)**가 **앱 사용**  
**의향에 긍정적 반응**을 보임

## 개발 앱 소개 Introduce To Development Application

### 개발 앱 주제 : 공공 클래스 통합 플랫폼 앱 [ON-Class]



**앱을 통해 언제, 어디서나 쉽고 간편하게 이용할 수 있음**



**유저 커뮤니티 기능으로 꿀팁, 정보 등 공유와 소통 가능**



**직관적이고 심플한 UI/UX로 누구나 쉽고 편하게 이용 가능**

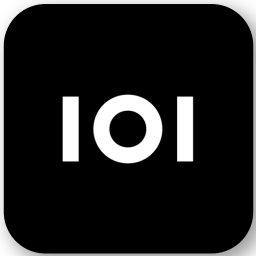


**강의별 평점 및 리뷰로 원하는 클래스를 한 눈에 비교 가능**



**퀴즈 기능을 통하여 학습 내용을 복습, 실력 점검 가능**

# 개발 앱 소개 Introduce To Development Application



## 유사 앱 비교

| 앱           | 클래스101             | ON-Class(온클래스)          |
|-------------|--------------------|-------------------------|
| 온라인 강의      | ○                  | ○                       |
| 다양한 분야의 클래스 | ○                  | ○                       |
| 구독형 서비스     | ○                  | X(국비를 활용한 무료 강의)        |
| 유저 커뮤니티     | ○                  | ○ (유저 커뮤니티로 학생간의 소통 가능) |
| 강사의 평가      | X (강사의 클래스 정보만 제공) | ○ (유저가 직접 평점을 매기고 확인)   |
| 퀴즈 기능       | X (강의 영상만 제공함)     | ○ (퀴즈 기능을 통한 학습 실천 가능)  |

# 와이어프레임 Wireframe

## 디자인 스케치 - 1) 회원 가입

회원 가입 창

10:31

회원님에 대하여 알려주세요!

회원가입

ID

사용하실 아이디를 입력해주세요

중복확인

password

대문자, 소문자, 특수기호를 사용해주세요

password check

설정된 비밀번호를 한번 더 입력해주세요

E-mail

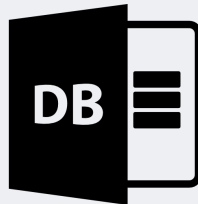
이메일을 입력해주세요

major

회원님의 전공을 선택해주세요

회원가입 완료

회원 정보를  
입력시 DB에  
회원 정보를 저장



DB에 기록된  
회원 정보로 로그인

로그인 창

10:31

오늘도 함께해주셔서 감사해요!

로그인

ID

아이디를 입력해주세요

password

비밀번호를 입력해주세요

Login

ID/비밀번호를 잊으셨나요?

DB의 정보와  
입력된 ID와  
비밀번호가  
일치시 로그인

# 와이어프레임 Wireframe

## 디자인 스케치 - 2) 강의 시청



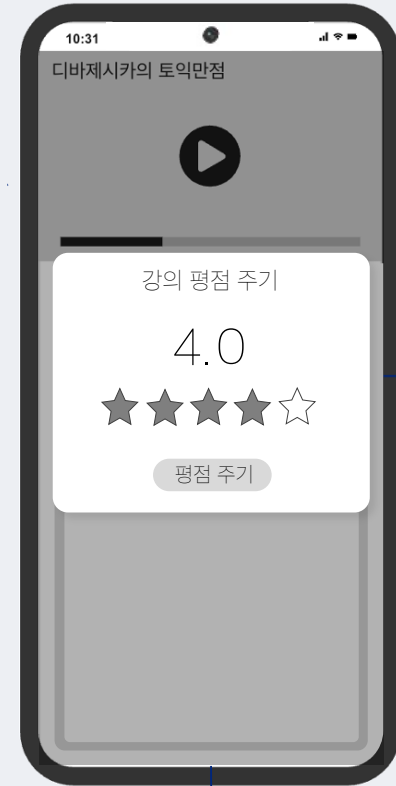
강의 시청 창

영상을 시청할 수 있는 화면

강의에 대한 평점을 볼 수 있으며  
평점을 주는 것 또한 가능

의견을 댓글로 작성 가능

퀴즈 기능으로 강의에 대한  
메모를 작성 가능



별점 평가 창

# 와이어프레임 Wireframe

## 디자인 스케치 - 3) 유저 커뮤니티

커뮤니티  
기능으로  
유저간의  
소통 가능

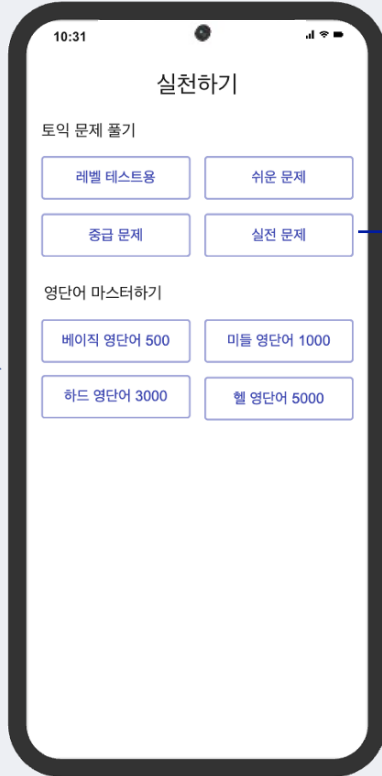


DB  
게시글 및  
댓글을  
DB에 저장  
↓  
페이징으로  
글검색 가능



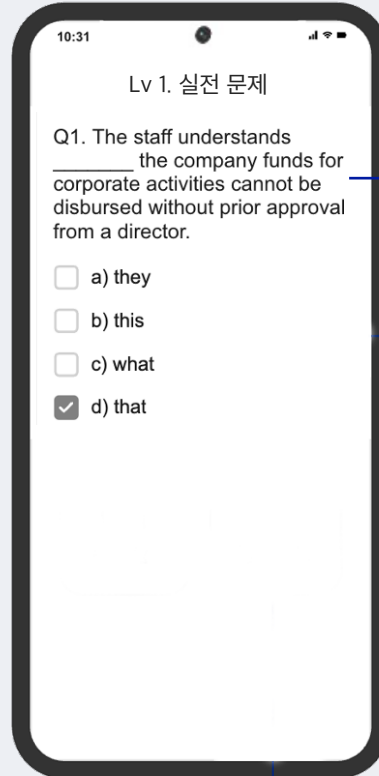
# 와이어프레임 Wireframe

## 디자인 스케치 - 4) 퀴즈 풀이



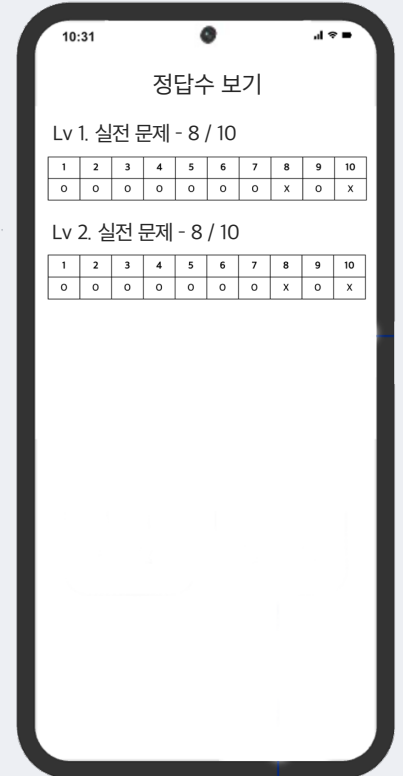
퀴즈 풀이 창

**퀴즈 풀이로  
강의내의 문제를  
한번 더 실천**



퀴즈 실천 창

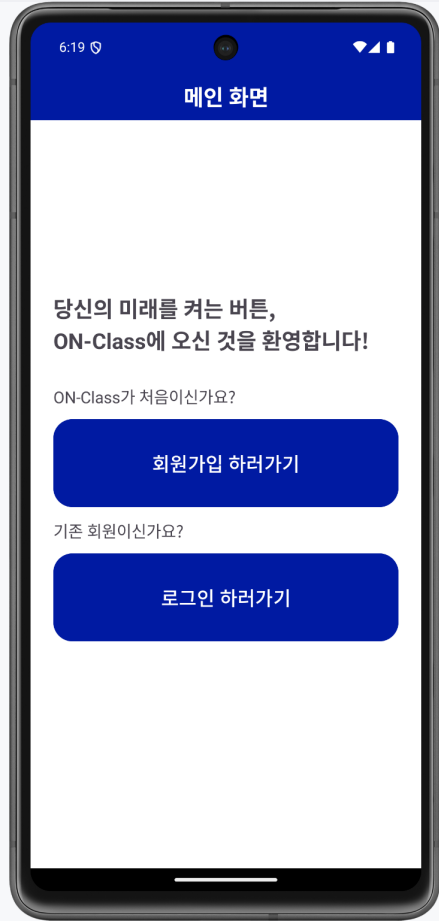
**문제 풀이 후  
정답 수를 체크해  
직관적으로 확인**



정답 확인 창

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 메인 화면 [Main Screen]



### MainAcitivity.java

```
package com.example.project;
```

```
import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
```

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button loginButton = findViewById(R.id.btn_login);
        Button signupButton = findViewById(R.id.btn_signup);

        loginButton.setOnClickListener(v -> {
            startActivity(new Intent(this, LoginActivity.class));
        });

        signupButton.setOnClickListener(v -> {
            startActivity(new Intent(this, SignupActivity.class));
        });
    }
}
```

메인에선 회원가입, 로그인의  
최소한의 기능만 적용

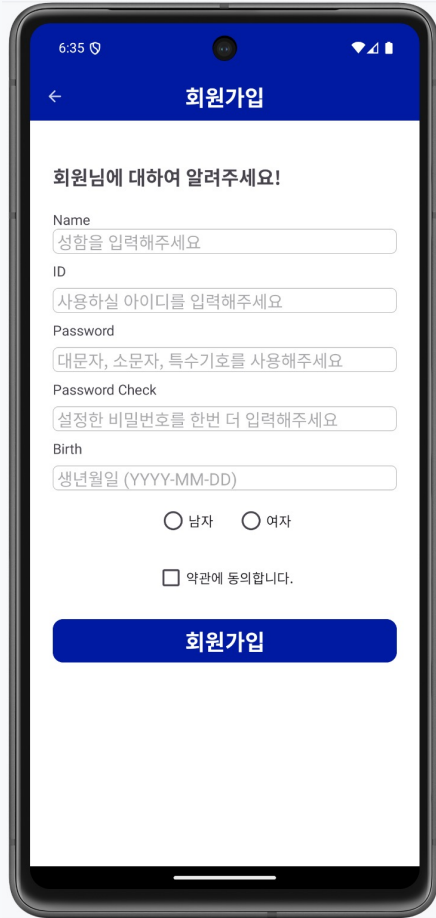
버튼 findViewById로 변수 선언

setOnClickListener로 버튼을 누를 시  
로그인 화면 LoginActivity.class로 이동

setOnClickListener 버튼을 누를 시  
회원가입 화면 SignupActivity.class로 이동

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 회원 가입 [Signup]



### SignupActivity.java

```
dbHelper = new UserDatabaseHelper(this);

signupButton.setOnClickListener(v -> {
    String name = nameEditText.getText().toString().trim(); ~ id,pw등의 정보 변수 선언

    if (id.isEmpty() || name.isEmpty() || password.isEmpty() || confirmPassword.isEmpty()
        || birth.isEmpty()) {Toast.makeText(this, "모든 항목을 입력해주세요.", Toast.LENGTH_SHORT).show();
        return; }

    if (!termsCheckBox.isChecked()) {
        Toast.makeText(this, "약관에 동의해주세요.", Toast.LENGTH_SHORT).show();
        return; }

    if (!password.equals(confirmPassword)) {
        Toast.makeText(this, "비밀번호가 일치하지 않습니다.", Toast.LENGTH_SHORT).show();
        return;}

    if (dbHelper.isUserIdExists(id)) {
        Toast.makeText(this, "이미 존재하는 아이디입니다.", Toast.LENGTH_SHORT).show();
        return;}

    boolean success = dbHelper.insertUser(id, name, password, birth, gender);
    if (success) {
        Toast.makeText(this, "회원가입 성공!", Toast.LENGTH_SHORT).show();
        finish();
    }
    else {
        Toast.makeText(this, "회원가입 실패. 다시 시도해주세요.", Toast.LENGTH_SHORT).show(); }
}
```

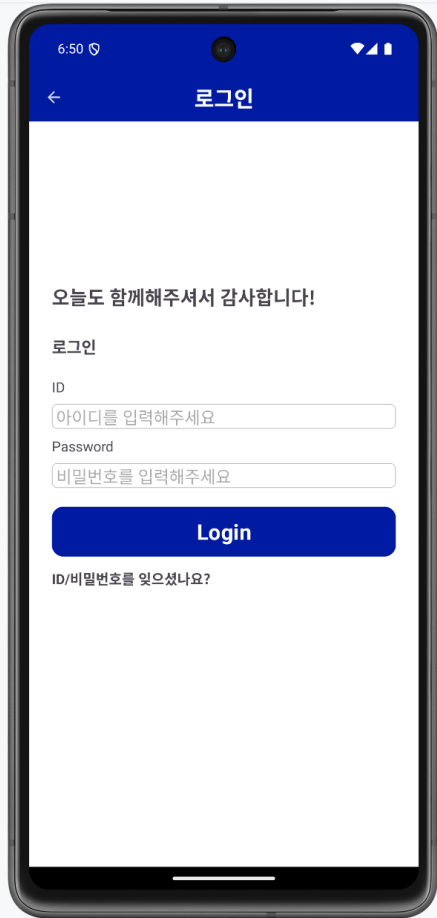
dbHelper를 사용  
이를 통해 DB에서  
사용자의 정보를 저장

if문을 활용하여  
정보가 제대로 입력되지  
않은 경우 오류를  
토스트 메시지로 변환함

**회원가입에선  
정보 입력 후  
DB에 정보를 전송**

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 로그인 [Login]



### LoginActivity.java

```
dbHelper = new UserDatabaseHelper(this);

loginButton.setOnClickListener(v -> {
    String id = idEditText.getText().toString();
    String password = passwordEditText.getText().toString();

    if (dbHelper.checkUser(id, password)) {
        Toast.makeText(this, "로그인 성공", Toast.LENGTH_SHORT).show();
        startActivity(new Intent(this, HomeActivity.class));
        finish();
    } else {
        Toast.makeText(this, "로그인 실패. 정보 확인해주세요.", Toast.LENGTH_SHORT).show();
    }

    if (dbHelper.checkUser(id, password)) {
        Toast.makeText(this, "로그인 성공", Toast.LENGTH_SHORT).show();

        SharedPreferences prefs = getSharedPreferences("user_prefs", MODE_PRIVATE);
        SharedPreferences.Editor editor = prefs.edit();
        editor.putString("user_id", id);
        editor.apply();

        startActivity(new Intent(this, HomeActivity.class));
        finish();
    }
});
```

### 로그인에선 DB에 저장된 회원정보와 일치하는지 확인

로그인 정보 확인 dbHelper가  
checkUser 유저의  
아이디, 비밀번호를 체크하고  
일치하면 로그인 성공  
다를 시 실패

로그인 성공시 홈화면  
HomeActivity로 이동

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 데이터베이스 응용 [Database]



### UserDatabaseHelper.java (1)

```
private static final String DB_NAME = "users.db";
private static final int DB_VERSION = 8;
```

→ 사용할 DB의 이름 정의  
및 DB 버전 설정  
(DB 버전 SQLite 8)

@Override

```
public void onCreate(SQLiteDatabase db) {
    db.execSQL("CREATE TABLE IF NOT EXISTS users (" +
        "id TEXT PRIMARY KEY, " +
        "name TEXT, password TEXT, birth TEXT, gender TEXT)");

    db.execSQL("CREATE TABLE IF NOT EXISTS UserCourses (" +
        "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
        "userId TEXT, " +
        "courseName TEXT)"); }
```

→ 데이터베이스 생성  
Users 테이블과  
UserCourses 테이블을  
생성함

```
public boolean insertUser(String id, String name, String password,
    String birth, String gender) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put("id", id);
    values.put("name", name);
    values.put("password", password);
    values.put("birth", birth);
    values.put("gender", gender);
    long result = db.insert("users", null, values);
    return result != -1;
}
```

→ 사용자 추가  
회원가입 시 추가되는  
데이터를 저장함

**DB의 전체 핵심 역할**  
**회원 정보를 저장 및 조회함**

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 데이터베이스 응용 [Database]



### UserDatabaseHelper.java (2)

```
public boolean checkUser(String id, String password) {
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor cursor = db.rawQuery("SELECT * FROM users WHERE id = ?
    AND password = ?", new String[]{id, password});
    boolean exists = cursor.getCount() > 0;
    cursor.close();
    return exists; }
```

로그인 시 사용자의 정보를  
체크하는 checkUser

```
public boolean isUserIdExists(String id) {
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor cursor = db.rawQuery("SELECT id FROM users WHERE id = ?",
    new String[]{id});
    boolean exists = cursor.getCount() > 0;
    cursor.close();
    return exists; }
```

회원가입 시 아이디의  
중복을 확인하는  
IsUserIdExists

```
public boolean registerCourse(String userId, String courseName) {
    if (isCourseRegistered(userId, courseName)) {
        return false;
    }
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put("userId", userId);
    values.put("courseName", courseName);
    db.insert("UserCourses", null, values);
    return true; }
```

수강신청 여부를 확인하여  
이 회원이 강의에 대하여  
중복 수강 신청을 막도록  
도와주는 registerCourse

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 홈화면 [Home]

### activity\_home.xml

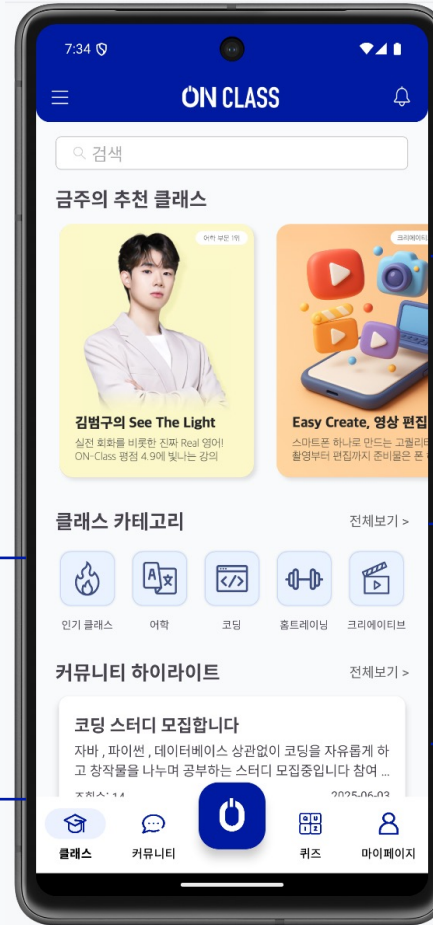
앱의 전체적인 UI/UX를  
돌보이는 홈화면

#### 클래스 카테고리 버튼

ImageView로 아이콘을 넣고  
어학, 코딩, 홈트레이닝 등의  
클래스 테마로 바로 이동

#### 하단 네비게이션 바

bottomNavigationView로  
주요 기능을 어디서든 사용  
가능한 네비게이션 바



#### 금주의 추천 클래스

RecyclerView를 활용하여  
이미지 슬라이드 가능 및  
페이지 클릭시 화면으로 이동

#### 전체보기 버튼

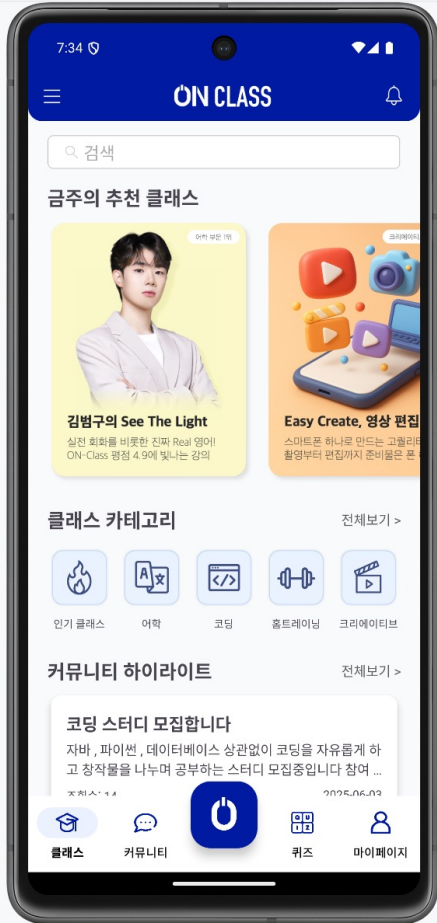
TextView에 클릭 이벤트를  
넣어 기능 활용

#### 커뮤니티 하이라이트

커뮤니티에서 조회수가 높은  
게시글 상위 3개를 보여줌

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 홈화면 [Home]



### HomeActivity.java

```

PostDatabaseHelper dbHelper = new PostDatabaseHelper(this);
List<Post> topPosts = dbHelper.getTopViewedPosts(3);

hotPostRecycler = findViewById(R.id.recycler_view_hot_posts);
hotPostRecycler.setLayoutManager(new LinearLayoutManager(this));
PostAdapter adapter = new PostAdapter(topPosts, post -> {
    Intent intent = new Intent(HomeActivity.this, PostDetailActivity.class);
    intent.putExtra("postId", post.getId());
    startActivity(intent); });
hotPostRecycler.setAdapter(adapter);

BottomNavigationView bottomNavigationView = findViewById(R.id.bottom_navigation);
bottomNavigationView.setOnItemSelectedListener(item -> {
    if (item.getItemId() == R.id.nav_class) {
        Intent intent = new Intent(HomeActivity.this, CategoryActivity.class);
        startActivity(intent); return true;
    } else if (item.getItemId() == R.id.nav_community) {
        Intent intent = new Intent(HomeActivity.this, CommunityActivity.class);
        startActivity(intent); return true;
    } else if (item.getItemId() == R.id.nav_quiz) {
        Intent intent = new Intent(HomeActivity.this, QuizActivity.class);
        startActivity(intent); return true;
    } else if (item.getItemId() == R.id.nav_mypage) {
        Intent intent = new Intent(HomeActivity.this, MyPageActivity.class);
        startActivity(intent); return true;
    }
    return false;
});
    
```

게시글도 PostDB처리를  
통하여 조회수 상위 3개를  
PostDatabaseHelper  
클래스에서 가져옴

PostAdapter 연결 후  
가져온 게시글을 아이템 뷰에  
바인딩하여 보여줌

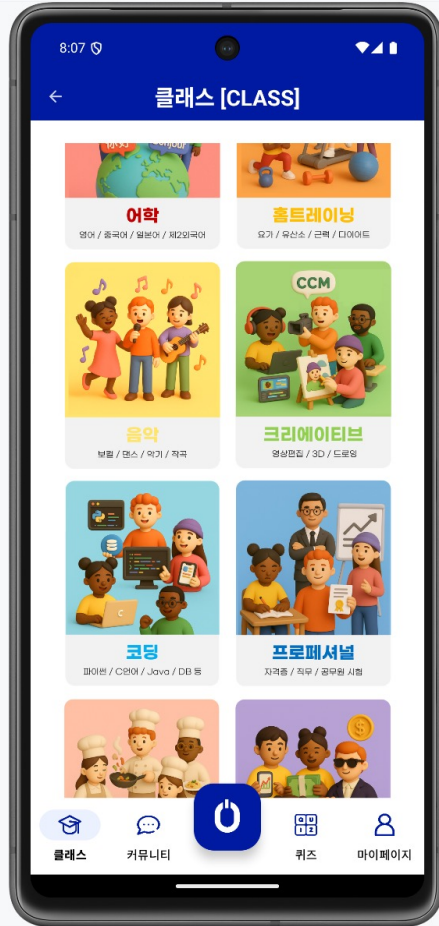
하단 네비게이션 바의 버튼  
setOnItemSelectedListener로  
버튼을 누를 경우에  
클래스, 커뮤니티, 퀴즈,  
마이페이지로 이동하는  
기능을 제공함

최대한 정보를 한눈에 보는 홈



# 프로토타이핑 Prototyping

## 최종 완성 설계 - 클래스 카테고리 [Class]



### CategoryActivity.java

```
public class MyGridAdapter extends BaseAdapter {
    Context context;
    Integer[] classID={R.drawable.ct1,R.drawable.ct2,R.drawable.ct3,R.drawable.ct4,
    R.drawable.ct5,R.drawable.ct6,R.drawable.ct7,R.drawable.ct8,R.drawable.last};
    public MyGridAdapter(Context c){
        context = c; }

    @Override
    public View getView(int i, View convertView, ViewGroup parent) {
        ImageView imageView = new ImageView(context);
        imageView.setLayoutParams(new GridView.LayoutParams(500,600));
        imageView.setScaleType(ImageView.ScaleType.FIT_CENTER);
        imageView.setPadding(5,10,5,20);
        imageView.setImageResource(classID[i]);
        final int pos = i;

        imageView.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                if (pos == 0) {
                    Intent intent = new Intent(context, LanguageActivity.class);
                    context.startActivity(intent);
                } else {
                }
            }
        });

        return imageView;
    }
}
```

BaseAdapter를 상속받아  
GridView에 이미지를  
배치할 이미지는 ClassID로  
R.drawable을 통해 미리  
삽입해둔 이미지를 불러옴

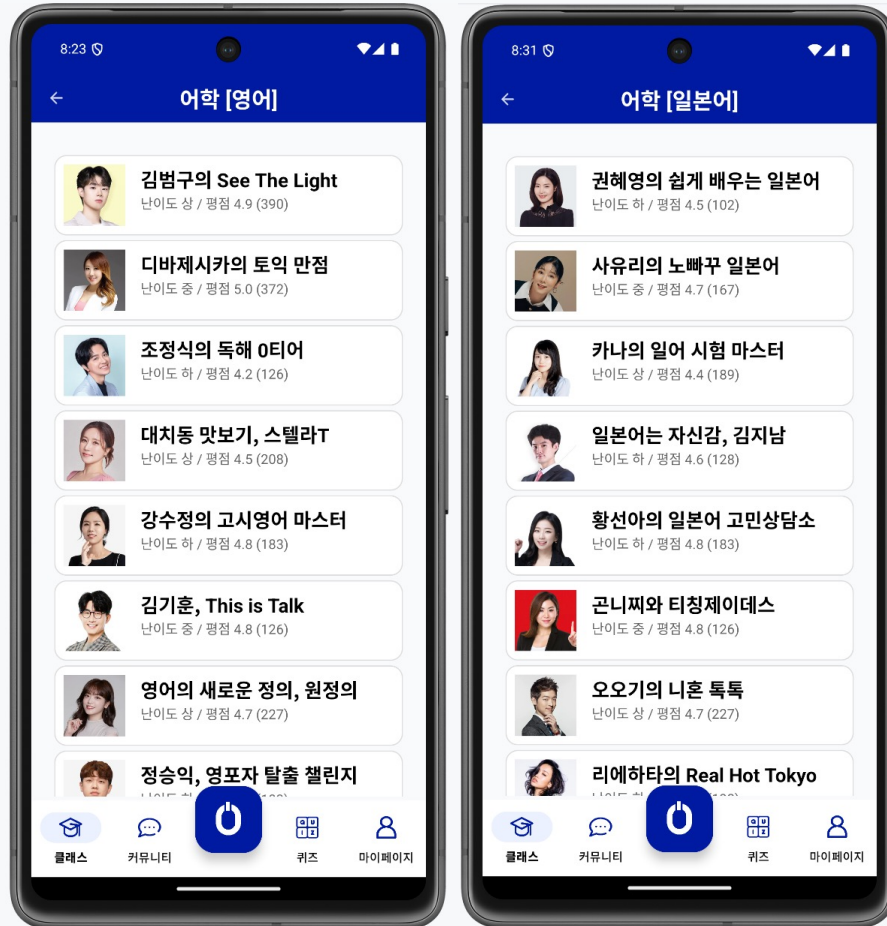
GridView 각 칸에 들어갈  
뷰를 반환함 ImageView를  
만들어 각 항목에 대하여  
이미지를 표시함(크기,정렬 등)

이미지 클릭 시 이벤트 처리  
setOnClickListener로  
첫 번째 이미지 (pos == 0)을  
클릭할 경우 어학 클래스  
LanguageActivity로 이동

**GridView를 적극 활용 후 이벤트 적용**

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 어학 클래스 [Language]



### LanguageActivity.java

```
viewPager = findViewById(R.id.viewPager);
viewPager.setAdapter(new LanguagePagerAdapter(this));
viewPager.registerOnPageChangeCallback
(new ViewPager2.OnPageChangeCallback() {
    @Override
    public void onPageSelected(int position) {
        super.onPageSelected(position);
        toolbarTitle.setText(titles[position]);
    }
});

private static class LanguagePagerAdapter extends
    FragmentStateAdapter {public LanguagePagerAdapter
    (@NonNull FragmentActivity fragmentActivity) {
        super(fragmentActivity);
    }

    @NonNull
    @Override
    public Fragment createFragment(int position) {
        switch (position) {
            case 0: return new EnglishFragment();
            case 1: return new JapaneseFragment();
            case 2: return new ChineseFragment();
            case 3: return new OtherLanguageFragment();
            default: return new EnglishFragment();
        }
    }
    @Override
    public int getItemCount() {
        return 4;
    }
}
```

viewPager2에 어댑터를  
연결하고 페이지가 전환될 때  
타이틀(어학[영어] 등)을  
업데이트함

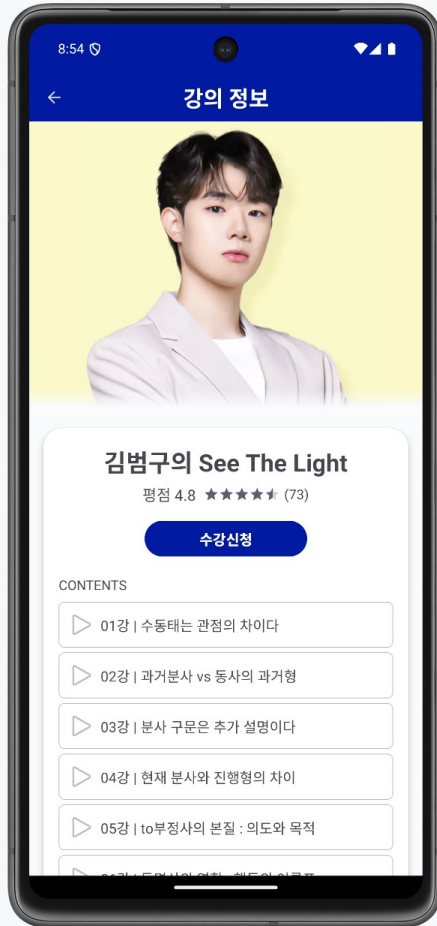
FragmentStateAdapter  
를 이용하여 각 페이지를 관리

어학 부문의 타이틀별(영어,  
일본어, 중국어, 제2외국어)로  
미리 생성해둔 Fragment를  
불러와 슬라이드로 넘겨볼 수  
있도록 호출함

ViewPager로 슬라이드 구현

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 강의 정보 보기 [Class Information]



### BeomguActivity.java

```
Button classRegBtn = findViewById(R.id.classregbtn);
classRegBtn.setOnClickListener(v -> {
    if (db.registerCourse(userId, "beomgu")) {
        Toast.makeText(this, "수강 신청이 완료되었습니다!", Toast.LENGTH_SHORT).show();
    } else { Toast.makeText(this, "이미 신청한 강의입니다.", Toast.LENGTH_SHORT).show();
    }
});
```

```
TextView lecture1 = findViewById(R.id.lecture1);
lecture1.setOnClickListener(v -> {
    if (db.isCourseRegistered(userId, "beomgu")) {
        Intent intent = new Intent(BeomguActivity.this, LectureDetailActivity.class);
        intent.putExtra("lectureNumber", 1);
        intent.putExtra("title", "제 1강 - 수동태는 관점의 차이이다");
        intent.putExtra("videoResId", R.raw.vedio1);
        startActivity(intent);
    } else { Toast.makeText(this, "수강 신청 후 이용할 수 있습니다.", Toast.LENGTH_SHORT).show();
    }
});
```

```
textRating = findViewById(R.id.textRating);
textReviewCount = findViewById(R.id.textReviewCount);
```

```
RatingDatabaseHelper ratingDb = new RatingDatabaseHelper(this);
float avgRating = ratingDb.getAverageRating(1);
int count = ratingDb.getRatingCount(1);
```

```
String ratingStr = String.format("평점 %.1f", avgRating);
String countStr = "(" + count + ")";
```

```
textRating.setText(ratingStr);
textReviewCount.setText(countStr);
```

수강신청 버튼을 누를 경우  
db.registerCourse를 호출  
Beomgu(강의코드) 강의를  
신청함

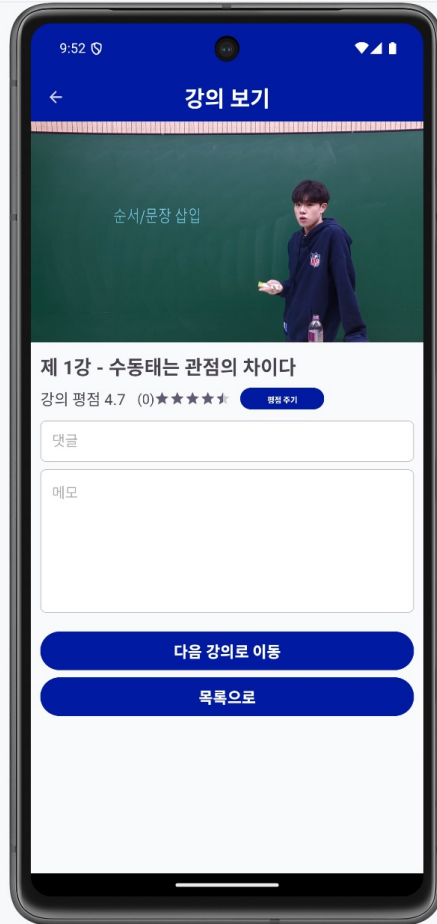
1강을 클릭했을 경우 사용자가  
강의를 신청했는지 확인후  
LectureDetailActivity로  
이동 미신청시 오류 메시지를  
토스트로 표시해줌

강의의 평점과 평점을 매긴 수를  
RatingDatabaseHelper  
객체를 통해 평점과 리뷰 수를  
가져와 표시해주는 역할을함

**수강신청 및 강의 정보를 볼 수 있게함**

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 강의 보기 [Lecture]



### LectureDetailActivity.java

```
Uri videoUri = Uri.parse("android.resource://"
+ getPackageName() + "/" + videoResId);
```

```
videoView.setVideoURI(videoUri);
videoView.setMediaController(new MediaController(this));
videoView.start();
```

```
buttonRate.setOnClickListener(v -> {
    float rating = ratingBar.getRating();
    ratingDb.saveRating(currentLecture, rating);
```

```
float avgRating = ratingDb.getAverageRating(currentLecture);
int count = ratingDb.getRatingCount(currentLecture);
Toast.makeText(this, "별점 " + rating + "점이 등록되었습니다!",
    Toast.LENGTH_SHORT).show();
```

```
buttonNextLecture.setOnClickListener(v -> {
    if (currentLecture < 12) {
        Intent nextIntent = new Intent(this, LectureDetailActivity.class);
        nextIntent.putExtra("lectureNumber", currentLecture + 1);
        nextIntent.putExtra("title", "제 " + (currentLecture + 1) + "강");
        nextIntent.putExtra("videoResId", getVideoResId(currentLecture + 1));
        startActivity(nextIntent);
    } else { Toast.makeText(this, "마지막 강의입니다.", Toast.LENGTH_SHORT).show();
```

### VideoView를 활용한 영상 시청 가능

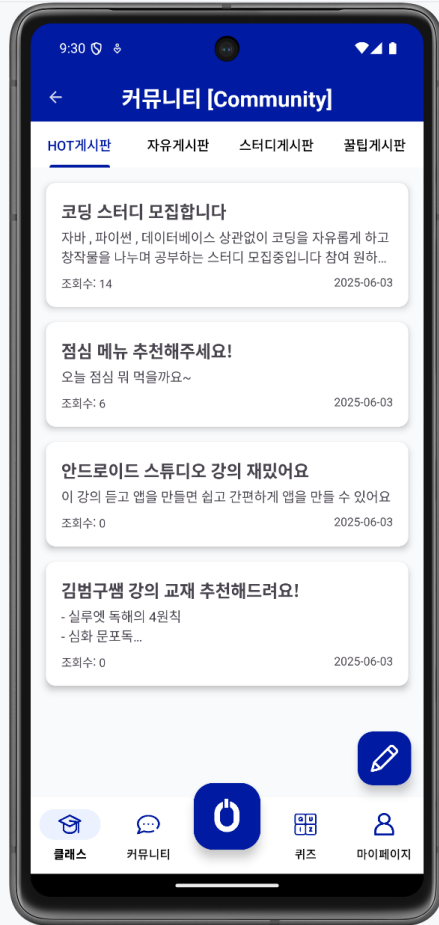
VideoView를 설정하여  
영상 경로 설정 후 영상 실행

RatingBar값을 가져와  
DB에 저장 후 평균 별점 및  
별점을 단 수를 업데이트하여  
볼 수 있게함

영상을 모두 시청하고 버튼을  
누를 시 다음 강의로 이동하고  
다음 강의를 없을 경우 마지막  
강의라는 토스트 메시지 출력

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 커뮤니티 [Community]



### CommunityActivity.java

```
fabWrite.setOnClickListener(v -> {
    Intent intent = new Intent(CommunityActivity.this, PostWriteActivity.class);
    startActivity(intent);
});
```

글쓰기 버튼 클릭시 글쓰기 창 PostWriteActivity로 이동

```
private static class BoardPagerAdapter extends FragmentStateAdapter {
    public BoardPagerAdapter(AppCompatActivity activity) {
        super(activity);
    }
```

```
@Override
public Fragment createFragment(int position) {
    switch (position) {
        case 0:
            return BoardFragment.newInstance("hot");
        case 1:
            return BoardFragment.newInstance("free");
        case 2:
            return BoardFragment.newInstance("study");
        case 3:
            return BoardFragment.newInstance("tip");
        default:
            return BoardFragment.newInstance("free");
    }
}
```

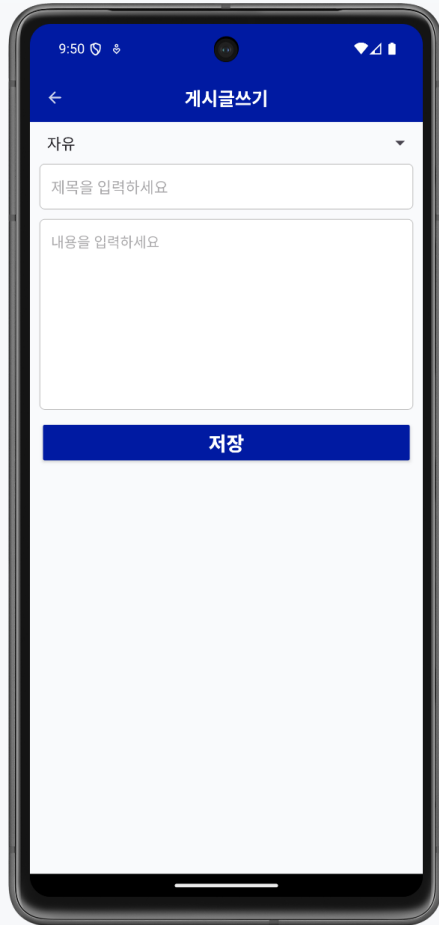
ViewPager2를 이용하여  
각 게시판에 따라서 보드를  
생성함 boardFragment  
게시판에서는 DB에서 글을  
불러와 보여줌  
PostDatabaseHelper

```
@Override
public int getItemCount() {
    return 4;
}
```

**ViewPager2를 사용해  
게시글 목록을 볼 수 있게함**

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 커뮤니티 [Community]



### PostWriteActivity.java

@Override

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_post_write);
```

```
    titleEditText = findViewById(R.id.editTextTitle);
    contentEditText = findViewById(R.id.editTextContent);
    categorySpinner = findViewById(R.id.spinnerCategory);
    saveButton = findViewById(R.id.buttonSave);
    dbHelper = new PostDatabaseHelper(this);
```

```
    saveButton.setOnClickListener(v -> {
        String title = titleEditText.getText().toString().trim();
        String content = contentEditText.getText().toString().trim();
        String selected = categorySpinner.getSelectedItem().toString();
        String category;
```

```
String date = new SimpleDateFormat("yyyy-MM-dd", Locale.getDefault()).format(new Date());
```

```
    if (title.isEmpty() || content.isEmpty()) {
        Toast.makeText(PostWriteActivity.this, "제목과 내용을 입력해주세요", Toast.LENGTH_SHORT).show();
        return;
    }
```

```
    Post post = new Post(0, title, content, category, date, 0, 0);
    boolean success = dbHelper.insertPost(post);
```

```
    if (success) {
        Toast.makeText(PostWriteActivity.this, "게시글이 저장되었습니다", Toast.LENGTH_SHORT).show();
        finish();
    } else { Toast.makeText(PostWriteActivity.this, "저장 실패. 다시 시도해주세요", Toast.LENGTH_SHORT).show();
```

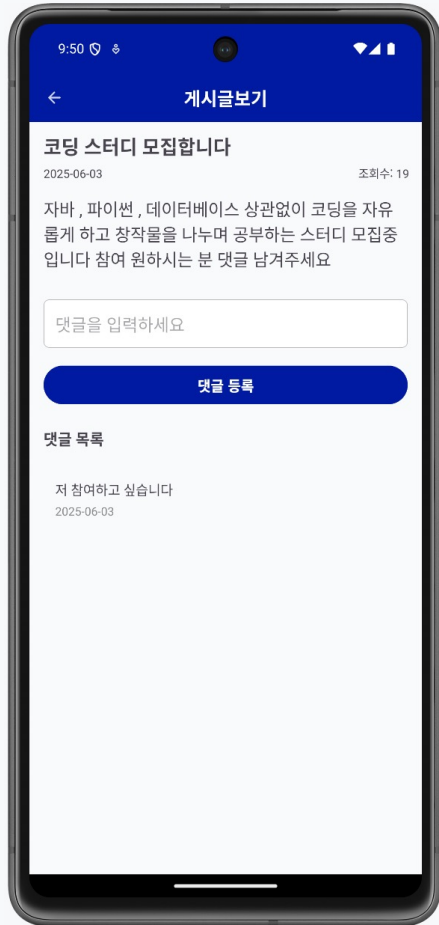
레이아웃 연결하여 뷰 바인딩  
XML의 레이아웃을 참조하고  
PostDatabaseHelper를  
참조하여 게시글 DB를 사용

저장버튼 누를 시 입력된 내용을  
저장하고 날짜를 기록함  
또한 게시글 업로드 여부에 따라  
토스트 메시지로 알림



# 프로토타이핑 Prototyping

## 최종 완성 설계 - 커뮤니티 [Community]



### PostDetailActivity.java

```

postId = getIntent().getIntExtra("postId", -1);
if (postId != -1) {
    dbHelper.increaseViews(postId);
    Post post = dbHelper.getPostById(postId);
    if (post != null) {
        titleTextView.setText(post.getTitle());
        contentTextView.setText(post.getContent());
        dateTextView.setText(post.getDate());
        viewsTextView.setText("조회수: " + (post.getViews() + 1));
    }
}

commentButton.setOnClickListener(v -> {
    String commentText = commentEditText.getText().toString().trim();
    if (!commentText.isEmpty()) {
        String now = new SimpleDateFormat("yyyy-MM-dd",
            Locale.getDefault()).format(new Date());
        Comment comment = new Comment(postId, commentText, now);
        boolean success = commentDbHelper.insertComment(comment);
        if (success) {
            commentEditText.setText("");
            Toast.makeText(this, "댓글이 등록되었습니다.", Toast.LENGTH_SHORT).show();
            loadComments();
        }
    } else {
        Toast.makeText(this, "댓글을 입력하세요.", Toast.LENGTH_SHORT).show();
    }
});

```

postId를 Intent로 받아와 해당 게시글의 정보를 DB에서 조회하여 TextView로 표시  
조회수는 dbHelper.increaseViews(postId)로 증가 처리

EditText에 작성한 댓글을 등록시 DB에 저장하고 저장 성공시 댓글 리스트 새로고침 loadComments를 호출하고 자동으로 작성일 추가

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 데이터베이스 응용 [Database]



### PostDatabaseHelper.java

@Override

```
public void onCreate(SQLiteDatabase db) {
    String sql = "CREATE TABLE posts (" +
        "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
        "title TEXT, " +
        "content TEXT, " +
        "category TEXT, " +
        "date TEXT, " +
        "views INTEGER, " +
        "likes INTEGER)";
    db.execSQL(sql);
}
```

```
public boolean insertPost(Post post) {
    SQLiteDatabase db = this.getWritableDatabase();
}
```

```
public List<Post> getPostsByCategory(String category) {
    List<Post> postList = new ArrayList<>();
    SQLiteDatabase db = this.getReadableDatabase();
}
```

```
public Post getPostById(int id) {
    SQLiteDatabase db = this.getReadableDatabase();
}
```

```
public void increaseViews(int postId) {
    SQLiteDatabase db = this.getWritableDatabase();
}
```

### DB에서 게시글을 저장 및 관리

Posts 테이블 생성을 통해  
게시글 아이디, 제목, 내용,  
게시판, 날짜, 조회수 등을  
기록할 수 있게함

insertPost로 게시글을  
DB에 저장

카테고리별 게시글 조회

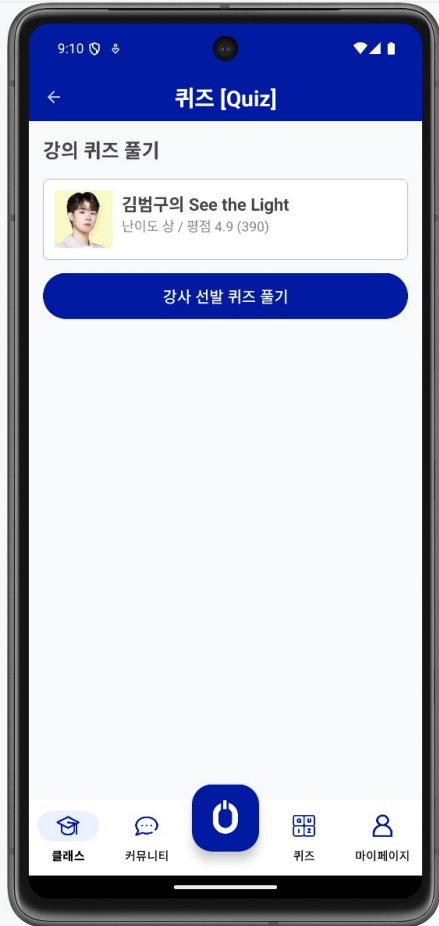
ID로 게시글 조회

로그인된 상태로 조회시  
조회수 +1 증가 변수



# 프로토타이핑 Prototyping

## 최종 완성 설계 - 퀴즈 [Quiz]



### QuizActivity.java

```
dbHelper = new UserDatabaseHelper(this);
```

```
SharedPreferences preferences = getSharedPreferences("user_prefs", MODE_PRIVATE);
currentUserId = preferences.getString("user_id", null);
```

```
courseInfoLayout = findViewById(R.id.courseInfoLayout);
courseTitle = findViewById(R.id.courseTitle);
courseDetail = findViewById(R.id.courseDetail);
courseImage = findViewById(R.id.courseImage);
btnStartQuiz = findViewById(R.id.btnStartQuiz);
noCourseMessage = findViewById(R.id.noCourseMessage);
```

```
private void loadQuizInfo() {
    Log.d("QuizActivity_Debug", "loadQuizInfo - currentUserId: " + this.currentUserId);

    if (this.currentUserId == null || this.currentUserId.isEmpty()) {
        Log.d("QuizActivity_Debug", "currentUserId is null or empty. Showing no courses.");
        showNoCourses();
        return;
    }
}
```

```
List<String> userCourses = dbHelper.getRegisteredCourses(this.currentUserId);
Log.d("QuizActivity_Debug", "User courses from DB: " + userCourses);
```

```
if (userCourses != null && !userCourses.isEmpty()) {
    String courseName = userCourses.get(0); // 첫 번째 강의만 표시
    Log.d("QuizActivity_Debug", "Course found: " + courseName + ". Showing course info.");
    showCourseInfo(courseName);
} else {
    Log.d("QuizActivity_Debug", "No courses found in DB for this user. Showing no courses message.");
    showNoCourses();
}
```

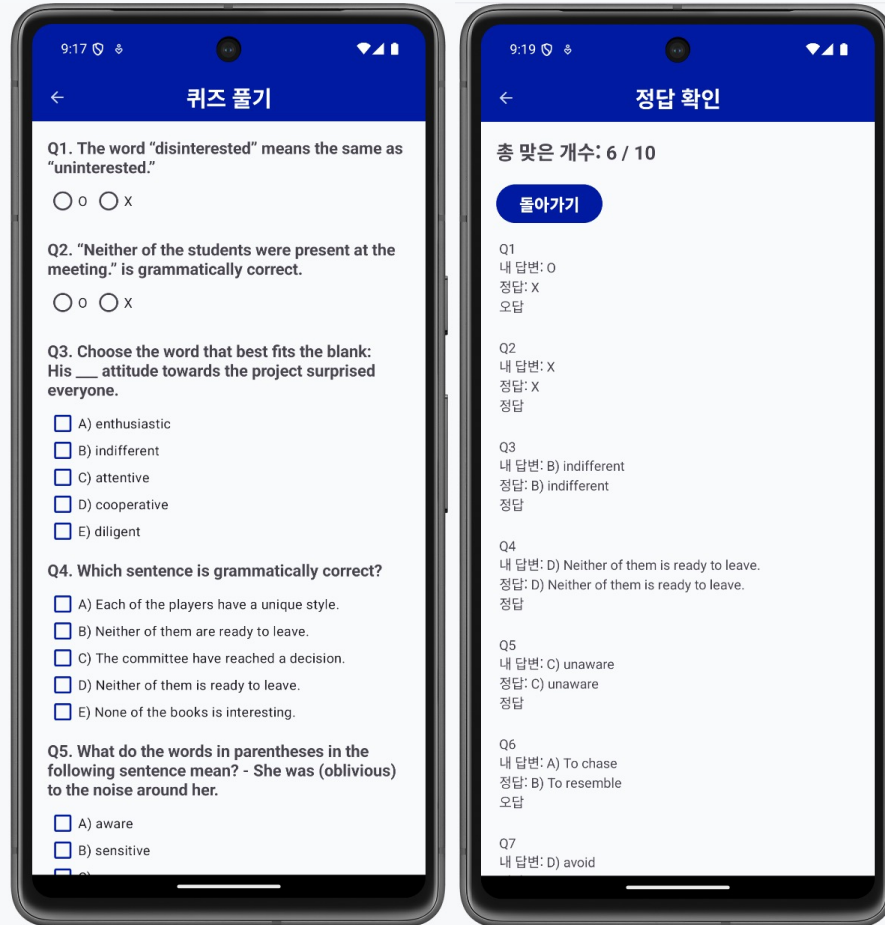
### 수강중인 강의의 퀴즈만 풀 수 있게함

DB를 사용하여 사용자 정보를  
SharedPreferences에서  
가져와 로그인 ID를 불러옴

loadQuizInfo 메소드를 통해  
DB에서 사용자의 수강 강의  
정보를 불러와 화면에 표시해줌  
즉, 수강 정보가 없을 경우  
퀴즈 창에 표시되지 않음

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 퀴즈 [Quiz]



### ClassquizActivity.java

```
private final String[] oxAnswers = {"X", "X"};
private final int[] choiceAnswers = {2, 4, 3, 2, 2};
private final String[] shortAnswers = {"sabbatical", "forbade", "compulsory"};
private String[] choiceAnswersText;

choiceAnswersText = new String[5];
for (int i = 0; i < 5; i++) {
    int correctIndex = choiceAnswers[i] - 1;
    int checkBoxId = getCheckBoxGroups()[i][correctIndex];
    AppCompatCheckBox cb = findViewById(checkBoxId);
    choiceAnswersText[i] = (cb != null) ? cb.getText().toString() : "(알 수 없음)";
}
```

RadioButton, CheckBox, EditText 등에 입력되는 값을 받아냄

### ResultActivity.java

```
int correctCount = 0; for (int i = 0; i < 10; i++) {
    String user = userAnswers.get(i); String correct = "";

    if (i < 2) { correct = oxAnswers[i];
    } else if (i < 7) { correct = choiceAnswersText[i - 2];
    } else { correct = shortAnswers[i - 7]; }
    boolean isCorrect = user.equalsIgnoreCase(correct);
    if (isCorrect) correctCount++;

    TextView resultText = new TextView(this);
    resultText.setText(String.format("%s\n내 답변: %s\n정답: %s\n%s\n",
        questionTitles[i], user, correct, isCorrect ? "정답" : "오답"));
    resultText.setPadding(8, 8, 8, 8);
    resultContainer.addView(resultText);
}
```

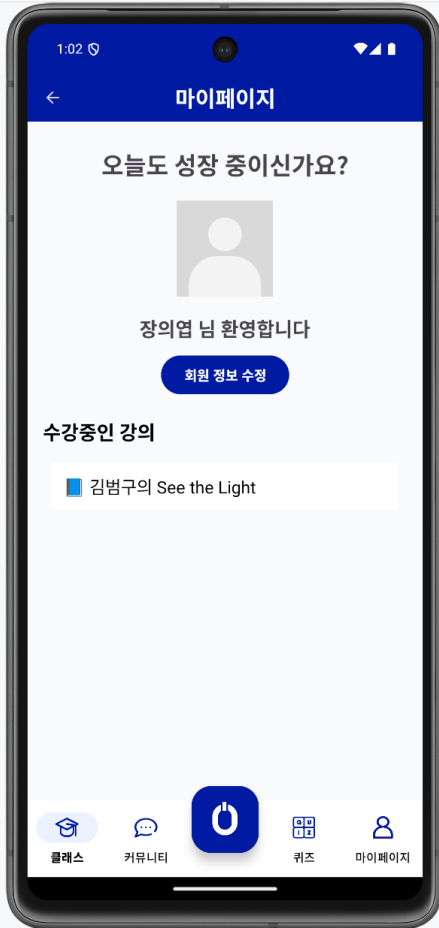
ClassquizActivity에서 입력된 정답과 일치한지 분석해 정답 수, 내 답변, 정답여부를 표시하여 출력해줌

```
TextView scoreText = findViewById(R.id.scoreText);
scoreText.setText("총 맞은 개수: " + correctCount + " / 10");
```

**퀴즈 풀이 및  
정답 출력을 구현함**

# 프로토타이핑 Prototyping

## 최종 완성 설계 - 퀴즈 [Quiz]



### MyPageActivity.java

```
SharedPreferences prefs = getSharedPreferences("user_prefs", MODE_PRIVATE);
userId = prefs.getString("user_id", "");
```

→ 사용자의 ID를 가져와 확인

```
List<String> courses = db.getRegisteredCourses(userId);
for (String course : courses) {
    View courseCard = createCourseCard(course);
    courseListLayout.addView(courseCard);}
}
```

→ 수강 중인 강의 목록을 가져와 강의 카드로 생성함

```
private View createCourseCard(String courseName) {
    TextView card = new TextView(this);
    card.setText(" " + convertCourseTitle(courseName));
    card.setTextSize(18);
    card.setTextColor(Color.BLACK);
    card.setBackgroundColor(Color.parseColor("#FFFFFF"));
    card.setPadding(30, 30, 30, 30);
}
```

→ 생성된 카드를 띄워 수강중인 강의 목록에 보이게함

```
LinearLayout.LayoutParams params = new LinearLayout.LayoutParams(
    ViewGroup.LayoutParams.MATCH_PARENT,
    ViewGroup.LayoutParams.WRAP_CONTENT
);
params.setMargins(0, 20, 0, 0);
card.setLayoutParams(params);
```

```
card.setOnClickListener(v -> {
    if (courseName.equals("beomgu")) {
        Intent intent = new Intent(MyPageActivity.this, BeomguActivity.class);
        startActivity(intent);}});
return card;
}
```

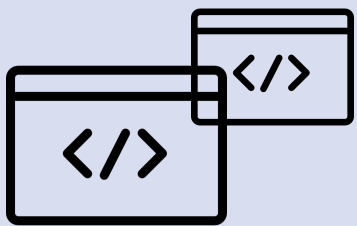
→ 강의 카드 클릭 시 바로 강의 페이지로 들어갈 수 있도록함

**내 정보 및 수강 목록을 확인 가능**

# AI 응용 내역 정리

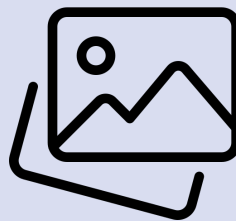
Ai application history summary

## 1) 코드 안정화 및 정리



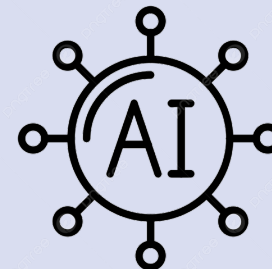
프로젝트 진행 중 점차 많은 기능들을 넣게 되며 작업의 양이 많아지자 코드의 오류가 발생하거나 앱 실행의 문제가 발생하는 경우가 잦았음 그럴때 로그캣이나 디버그의 기록을 Ai를 통해 점검하였을때 쉽게 오류를 발견할 수 있었음

## 2) 생성형으로 이미지 제작



아이콘, 로고, UI/UX등의 디자인은 직접 작업이 가능하였으나 클래스 카테고리에 들어갈 이미지 등 더욱 유저 친화적, 직관적인 인터페이스를 위한 이미지가 필요할 때 생성형 Ai로 만든 이미지를 넣어 작업함

## 3) 신기능 구현



수업 커리큘럼에 없는 내용의 기능을 구현하거나 응용할 때 최대한 인터넷, 블로그, 강의안 등을 활용하였으나 그럼에도 한계점이 있는 경우 Ai를 통해 기능을 구현 할 수 있는 코드를 생성한 뒤 이를 응용하여 결과물을 만들어냄

## 프로젝트를 통해 느낀 점

What I felt through the project



장의엽 20213211

<http://www.bu.ac.kr/20213211>

### 장의엽

앱 디자인 혹은 코딩을 통한 간단한 계산 등의 문제만 해결하기 위해 수업을 듣다가 직접적으로 앱을 만들 수 있는 것이 신기했습니다. 비록 짧은 시간 많은 양을 만들어내는 것이 쉽지 않았지만 프로젝트를 완성하고 나니 더욱 뿌듯했습니다. 처음엔 UI/UX 구현에 어려움도 있었으나 점차 개발 과정을 통해 새롭게 알게된 코드로 새로운 디자인, 기능 등을 구현할 수 있어 즐거웠습니다.



최지환 20203525

<http://www.bu.ac.kr/20203525>

### 최지환

학기 초에 세운 목표를 달성하면서 뿌듯함과 성취감을 느낄 수 있었습니다. 안드로이드 프로그래밍을 배우면서 새로운 언어와 도구를 익히는 과정이 도전적이었지만, 직접 앱을 만들어보며 재미있고 의미 있는 시간을 보낼 수 있었습니다. 프로젝트를 통해 문제를 해결하고 성취를 느끼며 개발에 대한 자신감을 얻을 수 있었습니다. 이번 수업과 프로젝트는 저에게 뜻깊고 성장할 수 있는 시간이었습니다.

**읽어주셔서 감사합니다**  
**Thank You**